Capstone Project Report

AUTOMATED MALWARE & THREAT DETECTION WORKFLOW USING N8N

TABLE OF CONTENTS

Abstract 3
1. Introduction 4
    1.1 Background 4
    1.2 Objectives 4
2. Methodology 5
    2.1 Tools and Technologies Used 5
    2.2 System Architecture 5
    2.3 URL and IP Analysis Procedure 6
    2.4 File Analysis Procedure 8
    2.5 Phishing Detection 8
    2.6 Result Consolidation via Merge Node 9
    2.7 Consolidated On-Screen Output 10
    2.8 Automated Telegram Delivery 11
3. Results & Analysis 12
    3.1 Analysis of Clean Results 12
    3.2 Analysis of Malicious-File Detection 13
    3.3 Consolidated Verdicts 14
    3.4 Phishing Detection Results 15
    3.5 Full End-to-End Consolidated Test 15
4. Discussion 17
5. Challenges & Limitations 17
    5.1 Challenges Encountered 17
    5.2 Limitations 18
6. Recommendations 19
7. Conclusion 19
8. References 20
9. Appendices 20
    Appendix A: Full Workflow Node Listing 20
    Appendix B: Sample Raw API Responses 21
    Appendix C: Key n8n Expressions Used 22
    Appendix D: Test Sample Reference 23

ABSTRACT

This report documents the design, implementation, and testing of an automated security triage workflow built using n8n, an open-source workflow automation platform.

The system accepts a URL and/or a file from a user through a web form and automatically evaluates it across four threat-intelligence dimensions — URL reputation, IP reputation, file hash reputation, and phishing indicators — using the VirusTotal and AbuseIPDB APIs alongside a custom rule-based phishing heuristic.

Results from all branches are consolidated using an n8n Merge node and delivered to the user through two channels simultaneously: an on-screen completion message and an automated Telegram notification.

The workflow was implemented, debugged, and validated using clean and known-malicious test samples, including the industry-standard EICAR test file, and a constructed phishing-style URL designed to trigger the new heuristic check.

This report presents the full methodology, results, design decisions, and challenges encountered.

1. INTRODUCTION

Cybersecurity threats such as malicious URLs, compromised IP addresses, and malware-laden files remain among the most common vectors used to compromise users and organisations. Manually checking a suspicious indicator against multiple threat-intelligence sources is slow and error-prone when performed by hand for every submission. This project addresses that problem by building an automated workflow that performs this triage automatically.

1.1 Background

Security teams routinely need to check whether a URL, IP address, or file is malicious before acting on it. Traditionally this requires manually visiting several separate threat-intelligence websites (such as VirusTotal or AbuseIPDB), submitting the indicator to each one individually, and interpreting the technical output returned. This is slow, repetitive, and prone to human error, particularly when a large number of indicators need checking. Workflow automation platforms such as n8n make it possible to orchestrate several such API calls together into a single, repeatable pipeline, removing the need for manual intervention at each step.

A further limitation of purely reputation-based checking is that it can only flag indicators that have previously been reported as malicious. A newly created phishing page — one that visually mimics a trusted brand's login page, for instance — often has no reputation history yet, and can return a "Clean" verdict despite being actively malicious. To address this gap, the workflow also includes a rule-based phishing/social-engineering check that inspects the URL's structure directly, rather than relying solely on external reputation databases.

1.2 Objectives

- Accept a URL and/or a file from a user through an automated intake form.
- Automatically determine whether a submitted URL is malicious using a reputable threat-intelligence API.
- Automatically determine whether the IP address associated with a URL has a history of abuse.
- Automatically determine whether an uploaded file matches a known malicious file signature.
- Automatically assess a submitted URL for common phishing/social-engineering indicators, independent of reputation history.
- Produce a clear, readable verdict (Clean / Malicious / Suspicious) for each check, without requiring the user to interpret raw API output.
- Consolidate all check results into a single, human-readable output shown to the user immediately upon submission.
- Automatically deliver the same consolidated result to a secondary channel (Telegram) without manual intervention.

2. METHODOLOGY

2.1 Tools and Technologies Used

- n8n (Cloud) — workflow automation platform used to build and run the entire pipeline.
- VirusTotal API — used for both URL reputation scanning and file hash lookups against 60+ antivirus engines.
- Google Public DNS API (dns.google) — used to resolve a submitted URL's domain to its underlying IP address.
- AbuseIPDB API — used to check the resolved IP address against a database of abuse reports.
- Telegram Bot API — used to deliver the consolidated result automatically to a Telegram chat via a bot created through BotFather.
- n8n Form Trigger, HTTP Request, Wait, Crypto, Edit Fields, Merge, and n8n Form (Form Ending) nodes — the core building blocks used throughout the workflow.

VirusTotal was selected over relying on a single antivirus engine because it aggregates results from over sixty independent vendors in one request. A single-engine check can miss a threat that vendor does not yet recognise, whereas VirusTotal's aggregated approach means a submission only needs to be missed by all sixty-plus engines simultaneously to produce a false negative — a substantially higher bar. This also allows the workflow to report a proportion (e.g. "22 of 61 engines flagged this"), which is a more informative signal than a single yes/no answer.

AbuseIPDB was selected for IP reputation because its abuse confidence score is derived from a large, continuously updated set of user-submitted abuse reports, rather than a static blocklist. A score-based model, rather than a binary blocked/not-blocked result, allows the workflow to distinguish between an IP with a handful of old, low-severity reports and one with many recent, high-confidence reports of active abuse.

Telegram was selected as a secondary delivery channel because it offers a free, simple bot API that can receive automated messages without requiring the recipient to have the n8n workflow open.

2.2 System Architecture

The workflow is triggered by a single form submission and splits into three parallel branches: URL/phishing analysis, IP analysis (derived automatically from the URL via DNS), and file analysis. All branches converge at a Merge node, whose combined output feeds two final destinations simultaneously: an n8n Form (Form Ending) node, which displays the consolidated result to the user on-screen, and a Telegram node, which sends the same result to a Telegram chat.

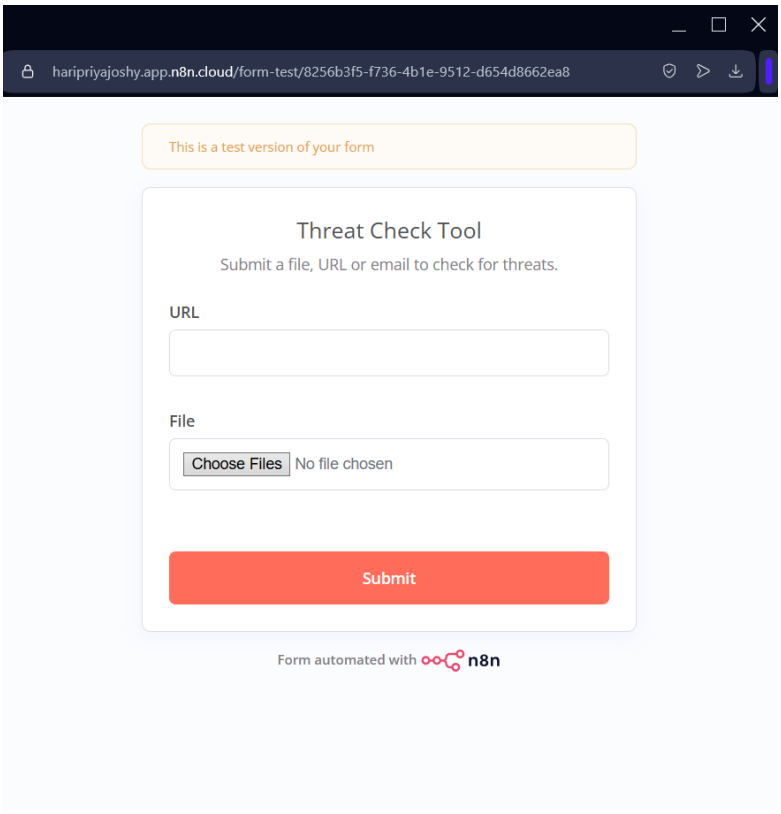


Figure 1: The web form used to submit a URL and/or file for analysis.

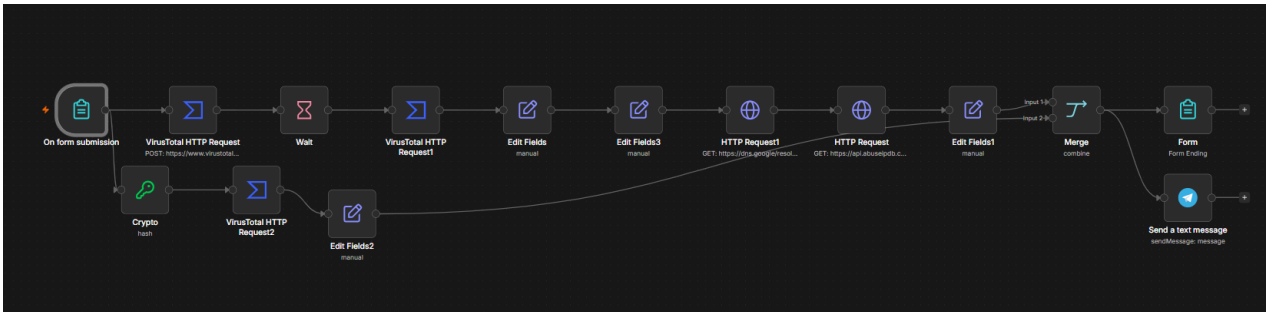


Figure 2: Complete updated workflow architecture.

2.3 URL and IP Analysis Procedure

- The submitted URL is sent to VirusTotal's URL-scanning endpoint, which queues the URL for analysis and returns an analysis ID.
- A timed wait step allows VirusTotal to complete its scan before the result is requested.
- The scan result is retrieved and summarised into a verdict, consisting of the scanned URL, the number of engines that flagged it as malicious, and a plain Clean/Malicious label.
- In parallel, the domain is resolved to an IP address using Google's public DNS resolution service. The resolution expression strips both the protocol (http:// or https://) and any path segment after the domain, ensuring a clean domain name is passed to the DNS lookup regardless of URL complexity.

- The resolved IP address is submitted to AbuseIPDB, and the returned abuse confidence score is summarised into a Clean/Suspicious label.

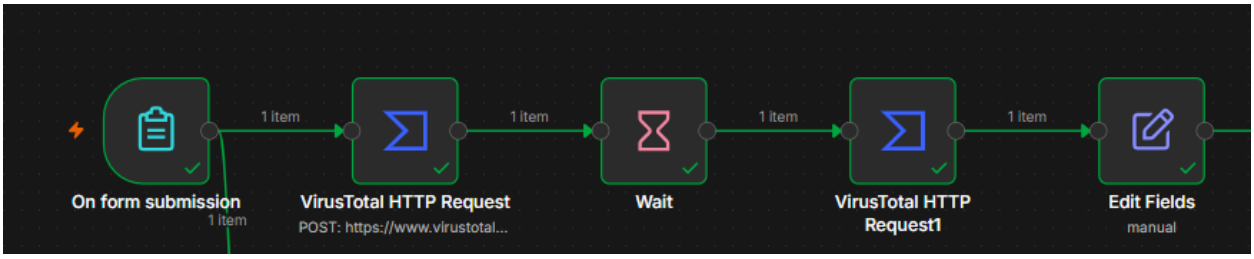


Figure 3: URL-checking pipeline executed successfully end-to-end.

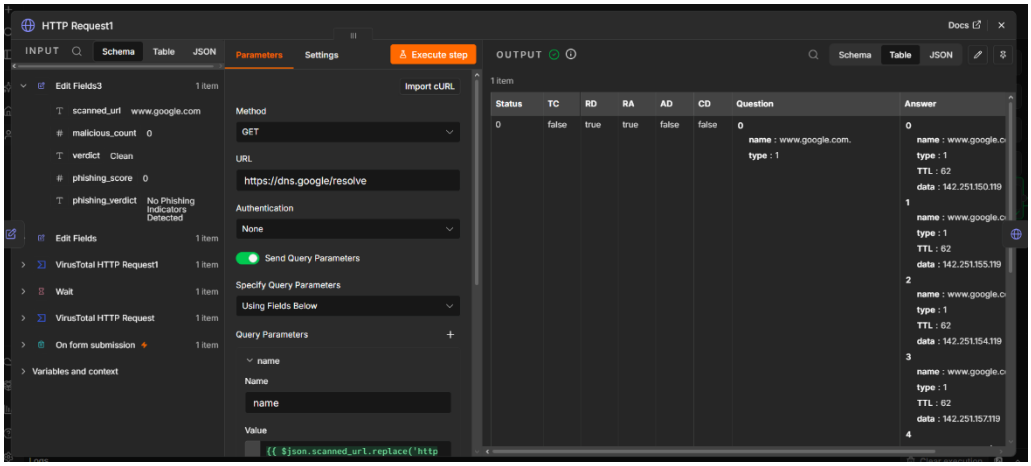


Figure 4: DNS lookup resolving a submitted domain to its IP address.

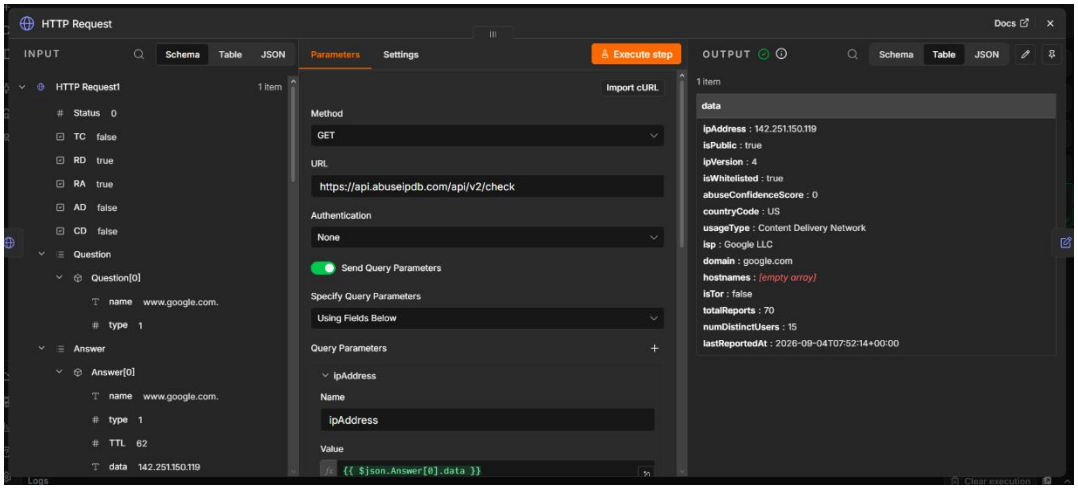


Figure 5: AbuseIPDB returning a reputation check for the automatically-derived IP address.

2.4 File Analysis Procedure

Files are not uploaded to VirusTotal directly. Instead, a SHA-256 hash — a unique fingerprint of the file's content — is generated locally within n8n using the Crypto node, and only this hash is sent to VirusTotal's file-lookup endpoint. This approach is faster and avoids transmitting file content unnecessarily, and works because VirusTotal maintains a database of hashes for previously analysed files.

An Edit Fields node was added immediately after the file-hash lookup to reduce VirusTotal's raw response to two clean fields — file_malicious_count and file_verdict — consistent with the consolidation already applied to the URL and IP branches.

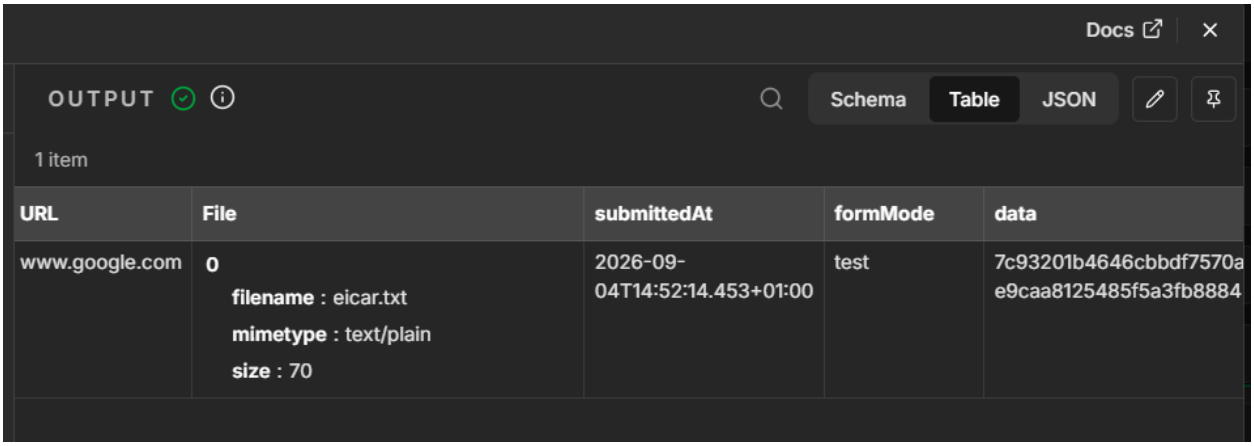


Figure 6: SHA-256 hash generated for an uploaded file using n8n's Crypto node.

2.5 Phishing Detection

A consolidation node was inserted immediately after the URL verdict step, implementing a rule-based phishing heuristic. Rather than relying on external reputation, this check inspects the structure of the submitted URL directly for six common phishing indicators, each contributing one point to a phishing_score:

- Presence of an "@" symbol in the URL.
- Use of a raw IP address in place of a domain name.
- Use of plain HTTP rather than HTTPS.
- More than three hyphens in the URL
- URL length exceeding 75 characters.
- Presence of common phishing lure keywords: login, verify, secure, account, update, confirm.

A total score of 2 or more is labelled "Possible Phishing Indicators"; below that threshold, "No Phishing Indicators Detected". This threshold, like the AbuseIPDB threshold, was chosen as a reasonable default rather than derived from a formal risk model.

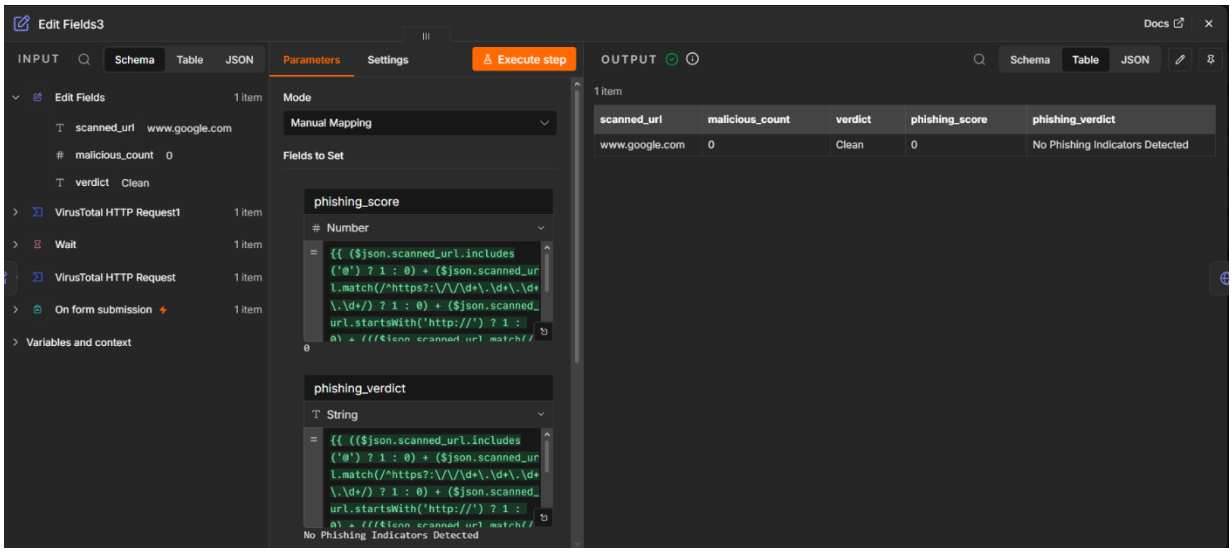


Figure 7: Phishing heuristic configuration.

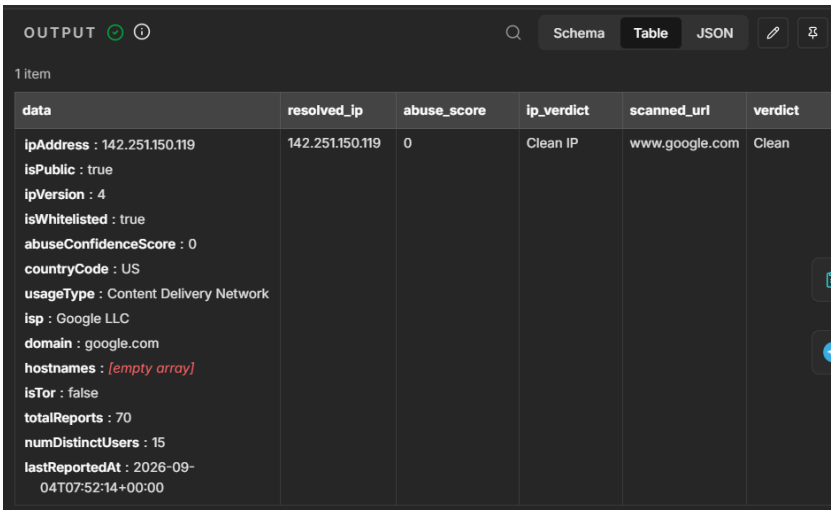
This check is intentionally independent of the VirusTotal URL verdict — a URL can correctly return "Clean" from VirusTotal (no prior malicious reports) while simultaneously being flagged "Possible Phishing Indicators" by this structural check, since the two checks measure different things: known reputation versus structural red flags.

2.6 Result Consolidation via Merge Node

Initially, each branch (URL, IP, file) produced its own verdict independently, with no single point in the workflow holding all results together. A Merge node (Mode: Combine, Combine By: Position) was added, taking the final output of all branches as separate inputs and combining them into a single item containing every field from every branch.

Two supporting fixes were required, documented in full in Section 5.1: HTTP Request nodes were found to silently drop upstream fields by default, requiring the "Include Other Input Fields" toggle to be enabled on downstream Edit Fields nodes; and several fields needed to be explicitly re-referenced from their original source node using n8n's $('NodeName') syntax, since intermediate API calls replaced the item's data entirely.

Figure 8:
Merge node combined output.



2.7 Consolidated On-Screen Output

The form trigger's own completion-message field was found to be structurally incapable of referencing data from later nodes in the workflow (n8n returns a "no path back to node" error, since the trigger sits before, not after, the nodes producing the results). The fix was to add a second, separate n8n Form node configured as "Form Ending", positioned immediately after the Merge node. Because this node is genuinely downstream of Merge, it can correctly reference the combined data, and its Completion Message displays all four results together.

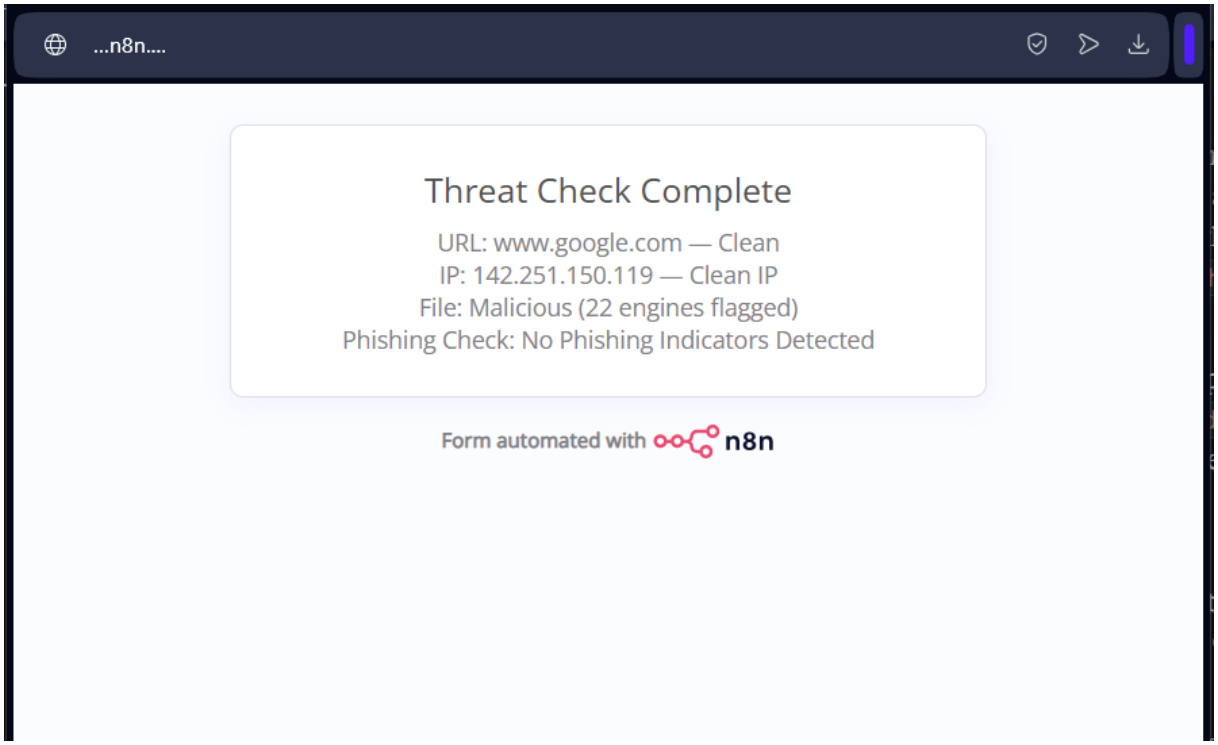


Figure 9: Consolidated final output shown to the user.

2.8 Automated Telegram Delivery

A Telegram bot was created via Telegram's official BotFather, and a Telegram "Send a text message" node was added as a second branch off the Merge node, running in parallel with the Form Ending node. Both nodes receive identical combined data, ensuring the on-screen result and the Telegram message never diverge. The bot's chat ID was obtained via Telegram's getUpdates API endpoint after an initial message was sent to the bot.

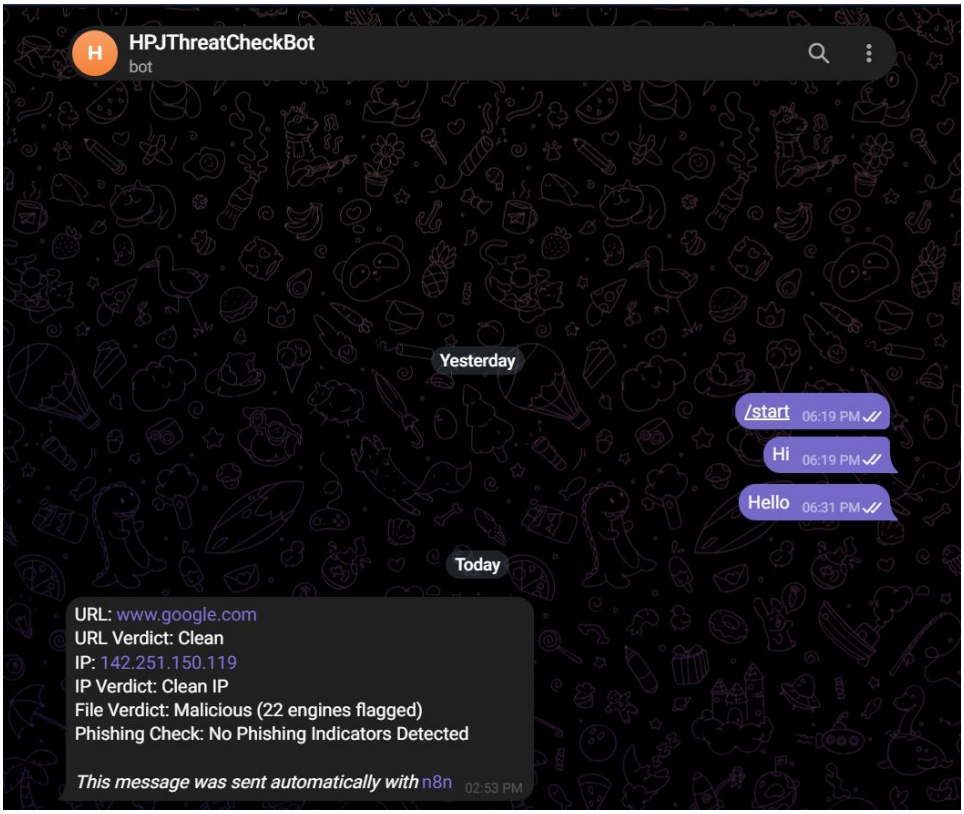


Figure 10: Automated Telegram delivery.

3. RESULTS & ANALYSIS

The workflow was tested with clean, known-malicious, and phishing-style samples to confirm correct differentiation across all four checks. A summary of test cases and outcomes is presented in Table 1.

CHECK TYPE	SAMPLE TESTED	TOOLS USED	RESULT
URL	google.com / wikipedia.org	VirusTotal	Clean (0-1/70+ engines)
IP Address	Resolved from URL	DNS + AbuseIPDB	Clean (low abuse score)
File (clean)	26-byte plain text file	SHA-256 + VirusTotal	Not previously seen
File (malicious)	EICAR test file	SHA-256 + VirusTotal	Malicious - 22/61 engines
Phishing (clean)	wikipedia.org	Rule-based heuristic	No indicators (score 0)
Phishing (flagged)	example.com/secure-login-...	Rule-based heuristic	Possible indicators (score 3)

3.1 Analysis of Clean Results

For clean inputs the workflow correctly returned low or zero malicious-engine counts and low abuse confidence scores. This is a meaningful result in itself, not simply an absence of a finding: both google.com and wikipedia.org are high-traffic, heavily scrutinised domains, and a false positive against either would be a strong indicator of a flawed detection pipeline. Correctly returning a clean verdict on well-established, legitimate domains therefore serves as a baseline correctness check for the whole URL-analysis branch, in the same way a negative control is used to validate an experimental method.

The IP-reputation check returned a low abuse confidence score for the IP addresses resolved from these clean domains. Because AbuseIPDB's score is built from independently submitted abuse reports rather than a static list, a low score here reflects an active absence of recent abuse reports, not merely that the IP is unlisted.

For file checking, the workflow correctly reported an unrecognised result for a novel, previously unseen 26-byte test file. This is expected and correct behaviour, since hash-based lookups can only identify files that have already been analysed by VirusTotal. This result demonstrates that the workflow behaves predictably and transparently even when a check cannot produce a positive-or-negative verdict, rather than failing silently or misreporting an unknown result as "clean".

3.2 Analysis of Malicious-File Detection

To validate malicious-file detection specifically, the industry-standard EICAR test file was used. This is a harmless test string that all major antivirus engines are designed to recognise and flag, making it a safe, standard way to confirm that a detection pipeline correctly identifies malicious content without handling real malware.

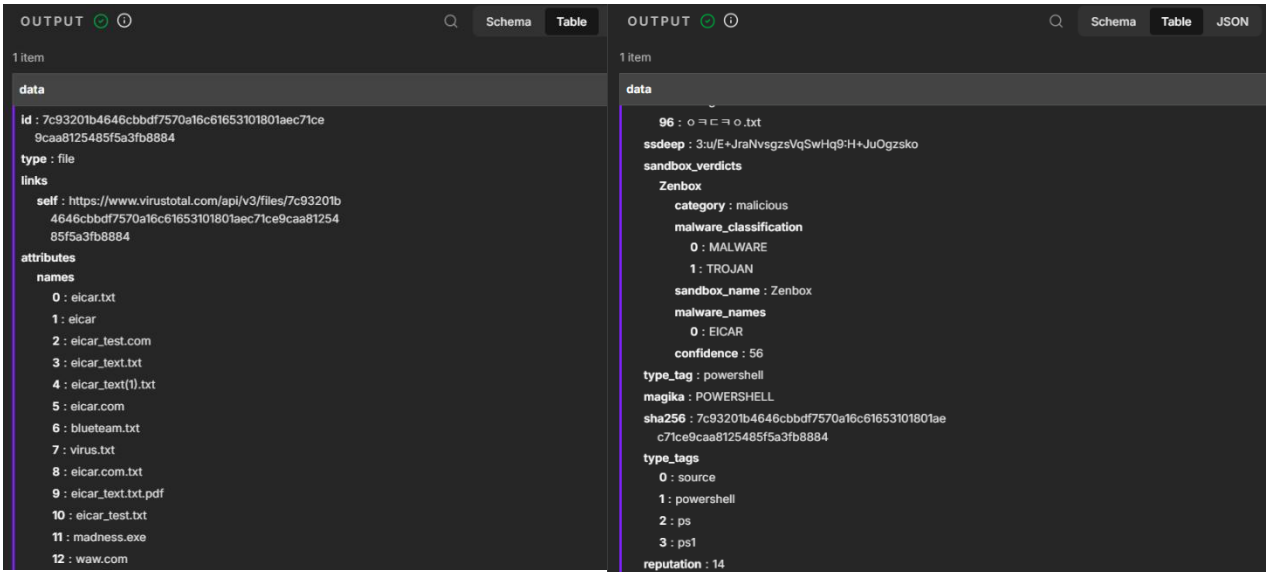


Figure 11: VirusTotal correctly identifying the EICAR test file as malicious.

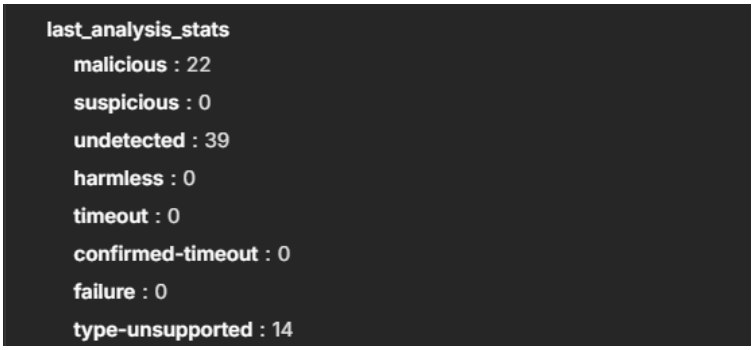


Figure 12: Detection statistics — 22 of 61 antivirus engines flagged EICAR.

Of the 61 engines that returned a verdict, 22 flagged the file as malicious, 39 returned undetected, and none returned suspicious or harmless. At first glance, a 22/61 detection rate might appear to indicate weak confidence; however, this is expected and correct behaviour for the EICAR file specifically. Because EICAR is a plain-text signature string rather than functional code, many general-purpose engines that rely on behavioural or heuristic analysis do not flag it, since it exhibits no malicious behaviour to analyse — only engines with a signature specifically defined for the EICAR string will match it. The sandbox verdict is the more significant signal here: VirusTotal's own sandbox explicitly classified the sample under the categories

MALWARE and TROJAN and identified it by name as EICAR, which confirms correct identification independent of the raw engine count.

3.3 Consolidated Verdicts

To avoid exposing raw API output to the end user, URL, IP, file, and phishing results are all consolidated into short, human-readable verdicts before being displayed or logged. The raw VirusTotal and AbuseIPDB responses each contain dozens of fields that are useful for a security analyst but would overwhelm a non-technical end user. Reducing each branch's output to simple fields makes the tool usable by someone with no security background, while the full underlying data remains available in the workflow for anyone who needs to inspect it further.

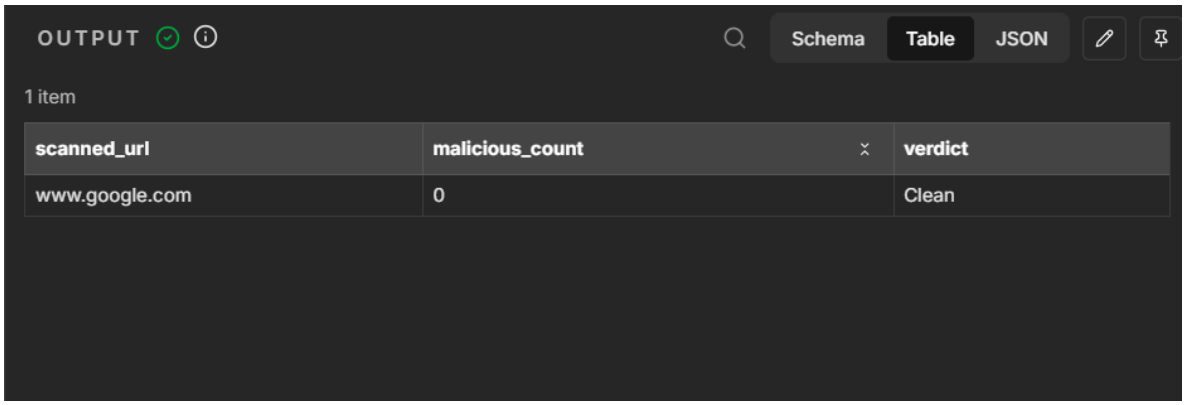


Figure 13: Consolidated URL verdict.

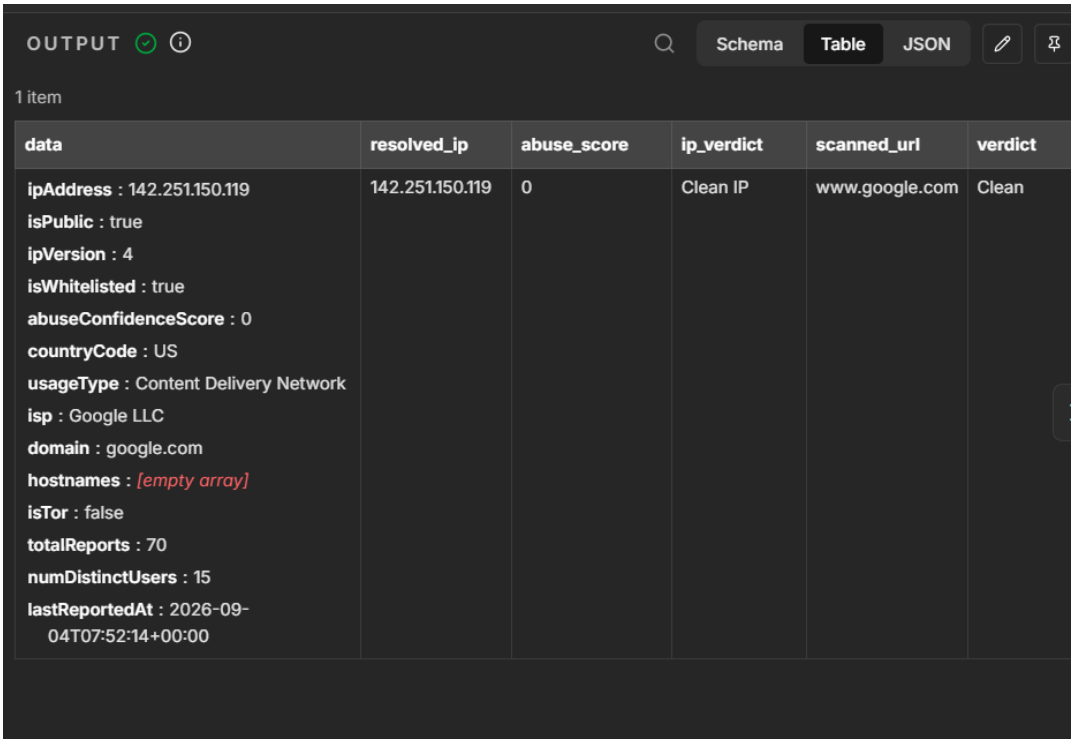


Figure 14: Consolidated IP verdict.

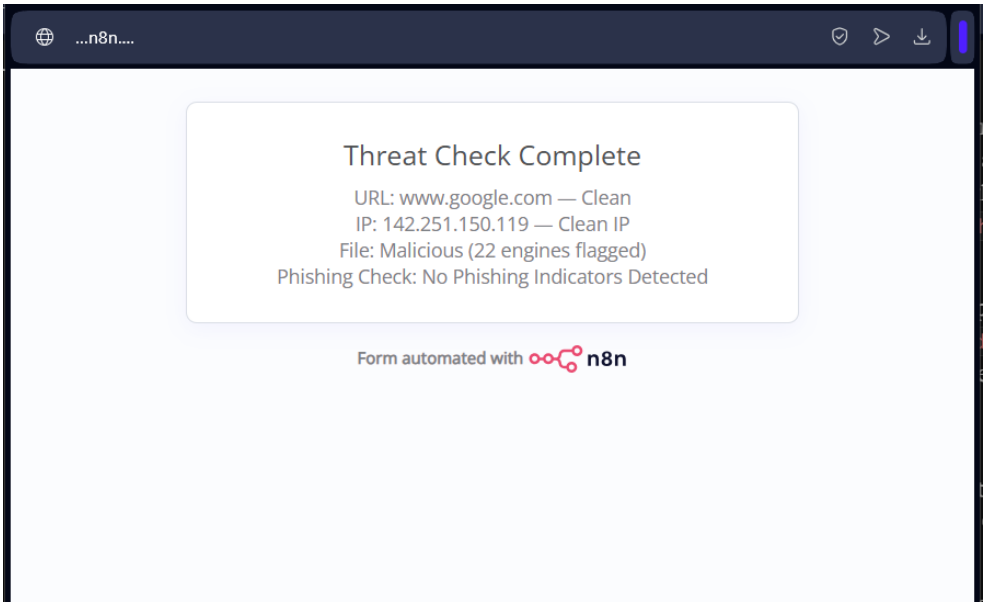
3.4 Phishing Detection Results

The constructed test URL used the reserved, safe example.com domain (officially allocated by IANA for documentation and testing) combined with a path containing multiple phishing-style red flags, allowing the heuristic to be validated without visiting or referencing any real malicious infrastructure. The result confirms the phishing check operates independently of VirusTotal's reputation verdict: the same URL returned "Clean" from VirusTotal (since example.com itself has no malicious history) while correctly being flagged "Possible Phishing Indicators" by the structural heuristic — demonstrating that the two checks provide genuinely different, complementary signal.

3.5 Full End-to-End Consolidated Test

A complete end-to-end test, submitting a phishing-style URL alongside the EICAR malicious test file, produced the following fully consolidated result, displayed identically on-screen and in Telegram:

This confirms all four checks — URL reputation, IP reputation, file reputation, and phishing structure — operate correctly together within a single automated run, with results reaching both delivery channels without any manual step between form submission and final output.



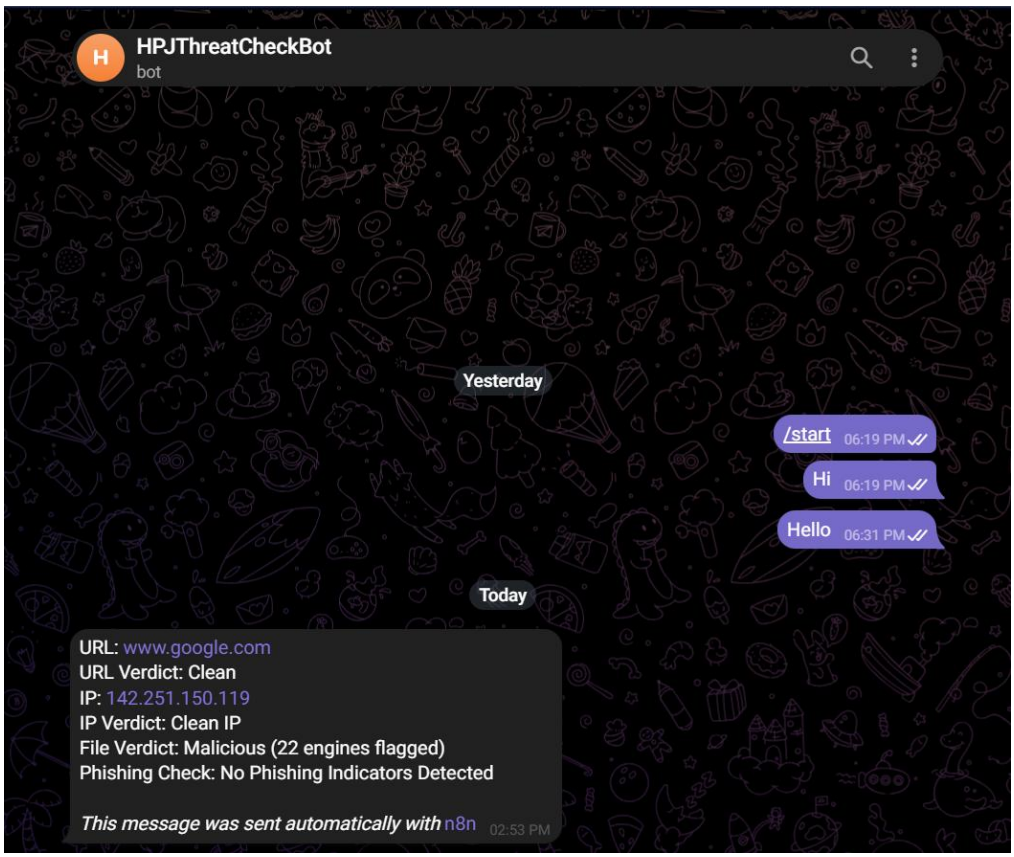


Figure 15: Full end-to-end consolidated test.

4. DISCUSSION

The results confirm that the workflow reliably distinguishes between clean and malicious/suspicious inputs across all four checked dimensions, and that each verdict remains explainable — traceable back to a specific, interpretable metric (engine count, abuse confidence score, phishing indicator count, or named malware classification) rather than an opaque decision. The use of a DNS-derived IP address, rather than a manually supplied one, proved an effective simplification: it reduced the number of inputs a user must provide while still producing a complete assessment.

A central design goal throughout was that automation should be visible as a single, consolidated outcome rather than scattered across separate branches. A single form submission produces one complete, combined verdict spanning URL, IP, file, and phishing analysis, delivered automatically and simultaneously to two independent channels — an on-screen message and a Telegram notification — with no manual step required by the user between submission and result.

The phishing/social-engineering check extends the project's value beyond pure reputation lookups. Reputation-based tools are inherently reactive — they can only flag what has already been reported. A structural heuristic, evaluated at submission time, catches some social-engineering patterns in brand-new URLs that have no reputation history yet, complementing rather than duplicating the reputation-based checks.

More broadly, the results suggest that meaningful security tooling can be assembled from free, publicly available services without writing detection logic from scratch — the workflow's value lies in orchestration (deciding what to check, in what order, how to combine results, and how to deliver them) rather than in the underlying detection technology itself, most of which is provided by third parties. The phishing heuristic is the one piece of genuinely original detection logic in the project, and is explicitly scoped as a simple, transparent rule-based check rather than a claim of comprehensive phishing detection. This is a realistic reflection of how many real-world security operations tools are built in practice.

5. CHALLENGES & LIMITATIONS

5.1 Challenges Encountered

A significant challenge arose during initial development of the URL-analysis retrieval step, which consistently returned a "resource not found" error despite the initial submission succeeding. Inspection of the raw outgoing HTTP request revealed that the request URL field was set to "Fixed" mode rather than "Expression" mode within n8n, causing the literal template text (e.g. {{ $json.data.id }}) to be sent instead of its evaluated value. Correcting the field to Expression mode resolved the fault. This debugging process demonstrated the value of inspecting raw request logs to isolate the true cause of an integration fault, rather than assuming the third-party API itself was at fault.

Several further integration challenges were encountered and resolved while implementing the consolidation, Telegram, and phishing features:

- Form trigger cannot reference downstream data: the form trigger's completion-message field returned a "no path back to node" error when referencing any node positioned after it in the workflow, since n8n's node-reference syntax requires the referenced node to be genuinely upstream. Resolved by adding a separate n8n Form (Form Ending) node positioned after the Merge node.
- HTTP Request nodes silently drop upstream fields: by default, HTTP Request nodes replace an item's data entirely with the API response, discarding any fields computed in earlier steps. This caused the URL verdict to disappear before reaching Merge. Resolved by enabling "Include Other Input Fields" on the relevant Edit Fields nodes, and explicitly re-referencing dropped fields using $('NodeName').item.json.fieldName where the toggle alone was insufficient.
- DNS resolution failure on URLs containing a path: the domain-extraction expression correctly stripped the http(s):// prefix but not any path segment after the domain, causing NXDOMAIN failures (and a cascading AbuseIPDB failure) when testing phishing-style URLs containing a path. Resolved by appending .split('/')[0] to the expression.
- Cross-field reference error within a single node: the phishing_verdict field's expression referenced a phishing_score field defined immediately above it in the same node — but n8n evaluates each field in Manual Mapping mode against the node's original incoming data, not against sibling fields being created in the same operation, causing it to always evaluate against undefined. Resolved by inlining the full scoring expression directly into the verdict field.
- Query-parameter format mismatch: the DNS lookup node's "Specify Query Parameters" setting was configured to expect a full JSON object rather than a plain string, causing a validation error. Resolved by switching to "Using Fields Below", matching the simpler Name/Value format already used successfully on the AbuseIPDB node.

5.2 Limitations

A limitation of the implementation is that file-hash lookups can only identify files previously analysed by VirusTotal; a genuinely novel malicious file with no analysis history would not be flagged by this method alone, and no dynamic or behavioural analysis is performed to compensate for this. Free-tier API rate limits also constrain how many checks can be performed in a given time period, which would need to be addressed for higher-volume or production use. The AbuseIPDB threshold used to distinguish Clean from Suspicious (an abuse confidence score above 25) was chosen as a reasonable default for this project rather than derived from a formal risk model, and could be tuned further with domain expertise or historical data. HaveIBeenPwned was excluded due to its API no longer being free, meaning compromised-email checking is not currently covered by this workflow.

The phishing heuristic is a simple, transparent rule-based scorer, not a machine-learning or visual-similarity check. It can be evaded by a URL that avoids all six specific structural patterns while still being malicious, and it can produce false positives on legitimate URLs that happen to contain a matched keyword or several hyphens. It is intended as a complementary signal alongside reputation checking, not a definitive phishing classifier.

6. RECOMMENDATIONS

- Supplement hash-based file lookups with dynamic or sandboxed analysis to detect genuinely novel malicious files with no prior analysis history.
- Add persistent logging (e.g. to a Google Sheet or database) so that all checks and verdicts are recorded for later review, rather than only being visible during a live run.
- Reconsider HaveIBeenPwned or an equivalent breach-checking service if budget allows, to extend coverage to compromised email addresses as originally scoped.
- Introduce rate-limit handling and queuing for higher submission volumes, to avoid exceeding free-tier API quotas in a production setting.
- Extend the phishing heuristic with a visual-similarity check (e.g. a URL-to-screenshot API) to catch brand-impersonation pages whose URLs do not match current structural rules but whose visual content mimics a known trusted site.
- Consider weighting phishing indicators rather than counting them equally, since some signals are stronger evidence than others.

7. CONCLUSION

This project successfully implements an automated, four-pronged threat-detection workflow using n8n and free-tier threat-intelligence APIs. A single user-facing form submission triggers checks across URL reputation, IP reputation, file hash reputation, and phishing/social-engineering structure, with results fully consolidated into a single, clear, non-technical verdict delivered simultaneously to two channels: an on-screen completion message and an automated Telegram notification.

Testing against clean, known-malicious, and phishing-style samples confirmed that the workflow produces accurate and correctly differentiated results across all four dimensions. Several genuine integration challenges were identified and resolved during development, each documented in Section 5.1, demonstrating a working understanding of n8n's data-flow model rather than surface-level node placement. The project satisfies the assignment's requirements while incorporating design refinements — automatic IP derivation, full result consolidation, dual-channel delivery, and an original rule-based phishing heuristic — that extend beyond the minimum expected scope

8. REFERENCES

VirusTotal. (2026). VirusTotal API v3 Documentation. Retrieved from https://docs.virustotal.com/

AbuseIPDB. (2026). AbuseIPDB API Documentation. Retrieved from https://docs.abuseipdb.com/

Google. (2026). Google Public DNS over HTTPS (DoH) API. Retrieved from https://developers.google.com/speed/public-dns/docs/doh

Telegram. (2026). Telegram Bot API Documentation. Retrieved from https://core.telegram.org/bots/api

n8n. (2026). n8n Documentation. Retrieved from https://docs.n8n.io/

European Institute for Computer Antivirus Research (EICAR). (n.d.). Anti-Malware Testfile. Retrieved from https://www.eicar.org/

IANA. (n.d.). Example Domains. Retrieved from https://www.iana.org/help/example-domains

9. APPENDICES

Appendix A: Full Workflow Node Listing

- On form submission (Trigger) — captures the submitted URL and, optionally, a file.
- VirusTotal HTTP Request (POST) — submits the URL to VirusTotal's URL-scanning endpoint.
- Wait — pauses execution (60 seconds) to allow the VirusTotal scan to complete.
- VirusTotal HTTP Request1 (GET) — retrieves the completed URL analysis result.
- Edit Fields — consolidates the URL result into a Clean/Malicious verdict.
- Edit Fields3 — computes phishing_score and phishing_verdict from the URL's structure.
- HTTP Request1 (GET) — resolves the submitted URL's domain to an IP address via Google's DNS API.
- HTTP Request (GET) — submits the resolved IP address to AbuseIPDB for a reputation check.
- Edit Fields1 — consolidates the IP result into a Clean/Suspicious verdict; re-includes URL and phishing fields.
- Crypto (parallel branch) — generates a SHA-256 hash of the uploaded file.
- VirusTotal HTTP Request2 (GET) — looks up the file hash against VirusTotal's file database.

- Edit Fields2 — consolidates the raw file-lookup response into file_malicious_count and file_verdict.
- Merge — combines the URL/IP branch and file branch outputs into a single item.
- Form (Form Ending) — displays the consolidated result to the user as the workflow's final on-screen response.
- Telegram (Send a text message) — delivers the same consolidated result automatically to a Telegram chat.

Appendix B: Sample Raw API Responses

B.1 — VirusTotal URL submission response:


```
{
  "data": {
    "type": "analysis",
    "id": "u-9d116b1b0c1200ca75016e4c010bc9483...",
    "links": { "self": "https://www.virustotal.com/api/v3/analyses/..." }
  }
}
```


B.2 — Google DNS resolution response:


```
{
  "Status": 0,
  "Answer": [ { "name": "wikipedia.org.", "type": 1, "TTL": 180, "data": "185.15.59.224" } ]
}
```


B.3 — AbuseIPDB reputation response (excerpt):


```
{
  "data": {
    "ipAddress": "185.15.59.224", "isPublic": true,
    "abuseConfidenceScore": 19, "countryCode": "NL",
    "usageType": "Content Delivery Network", "isp": "Wikimedia Foundation, Inc.",
    "domain": "wikimedia.org", "totalReports": 5, "isTor": false
  }
}
```

B.4 — VirusTotal file analysis response (excerpt, EICAR test file):


```
{
  "data": { "attributes": {
    "last_analysis_stats": { "malicious": 22, "suspicious": 0, "undetected": 39, "harmless": 0 },
    "sandbox_verdicts": { "Zenbox": {
      "category": "malicious",
      "malware_classification": ["MALWARE", "TROJAN"],
      "malware_names": ["EICAR"]
    } }
  } }
}
```


Appendix C: Key n8n Expressions Used

PURPOSE	EXPRESSION
Pass form URL to VirusTotal	{{ $json.URL }}
Retrieve VirusTotal analysis result	{{ $json.data.id }} (Expression mode)
Strip protocol and path before DNS lookup	{{ $json.scanned_url.replace('https://','').replace('http://','').split('/')[0] }}
Derive IP for AbuseIPDB	{{ $json.Answer[0].data }}
URL verdict logic	{{ $json.data.attributes.stats.malicious > 0 ? "Malicious" : "Clean" }}
IP verdict logic	{{ $json.data.abuseConfidenceScore > 25 ? "Suspicious IP" : "Clean IP" }}
File verdict logic	{{ $json.data.attributes.last_analysis_stats.malicious > 0 ? "Malicious" : "Clean" }}
Phishing verdict logic (inlined score)	{{ (six-factor sum, see Section 2.6) >= 2 ? "Possible Phishing Indicators" : "No Phishing Indicators Detected" }}

Re-including a dropped upstream field	{{ $('Edit Fields').item.json.scanned_url }}
Form Ending completion message	{{ $json.verdict }} / {{ $json.ip_verdict }} / {{ $json.file_verdict }} / {{ $json.phishing_verdict }}

Appendix D: Test Sample Reference

The EICAR standard antivirus test file used in Section 3.2 consists of the following 68-character ASCII string, saved as a plain text file. It is not executable code and poses no risk; it exists solely to trigger antivirus signature detection for testing purposes:

*X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H**

Phishing test URL used in Sections 2.6 and 3.4:

http://example.com/secure-login-verify-account-update-confirm-now

Constructed using the IANA-reserved, safe example.com domain, combined with a path deliberately containing multiple phishing-style structural indicators (plain HTTP, multiple hyphens, lure keywords) for controlled testing without referencing real malicious infrastructure.